

SOFTWARE ENGINEERING(UE24CS341A)

LAB-1

NAME: Arundhathi K

PES1UG24CS082

5-B

Database Query Performance Profiler Developer Tools & IT Operations

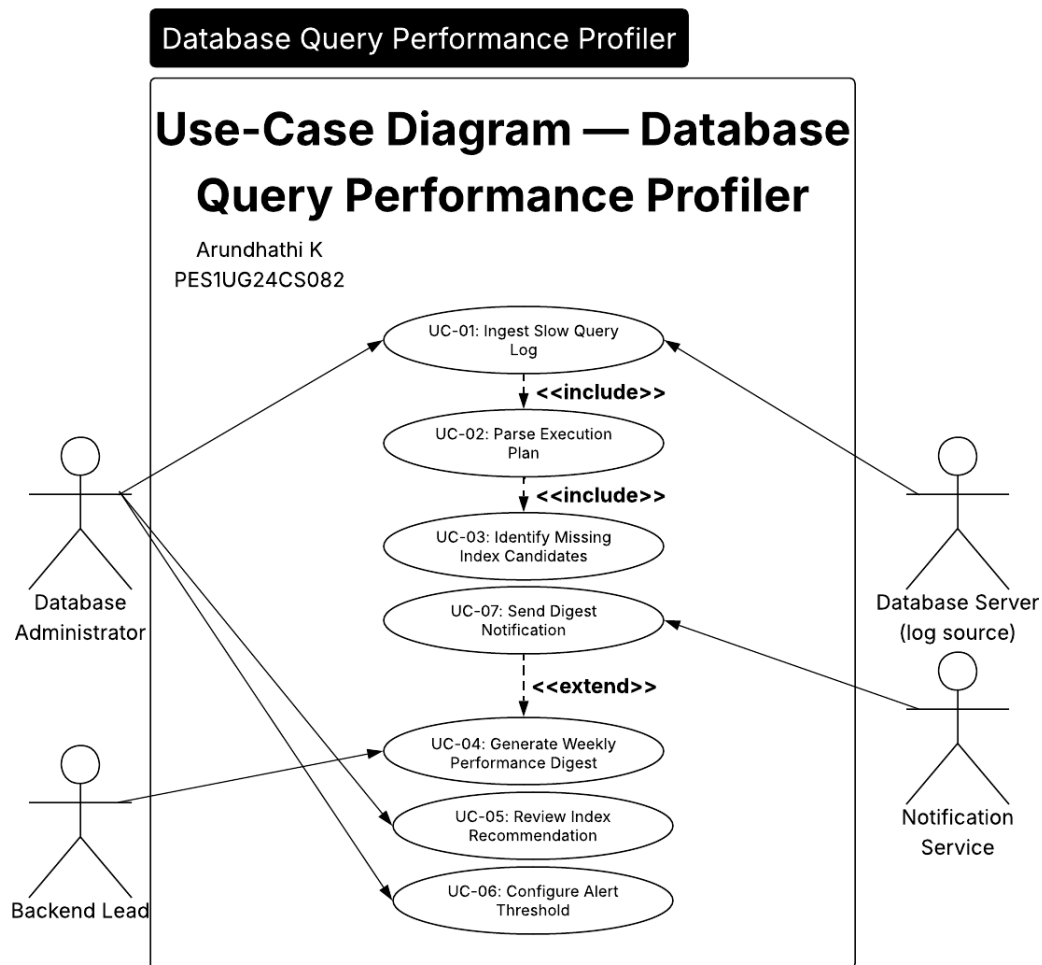
Functional and Non-Functional Requirements

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse PostgreSQL/MySQL EXPLAIN plans, highlight sequential table scans on large tables, and recommend optimal index column definitions.	High	Pass: A missing composite index is correctly identified on filtered columns. Fail: A query plan containing a sequential scan is incorrectly marked as optimized.	Core capability that transforms raw execution plans into actionable index recommendations for the DBA.
FR-002	Functional	The system shall ingest slow query logs from PostgreSQL and MySQL hosts at a configurable polling interval, with a default interval of 60 seconds, and store each entry with its query text, timestamp, and execution duration.	High	Pass: A slow query written to the log appears in the system's query store within two polling cycles, with all required fields populated. Fail: An entry is missing or contains incomplete fields	Log ingestion is the entry point for all downstream functionality, including profiling, indexing recommendations, and performance digests.

				after two polling cycles.	
FR-003	Functional	The system shall rank ingested slow queries by total time impact, calculated as execution time multiplied by frequency, and expose the top 20 offenders for each database instance.	Medium	Pass: The ranked list matches a manually computed execution time × frequency ranking for a seeded test dataset. Fail: The displayed order differs from the expected ranking.	DBAs need to prioritize queries based on overall performance impact rather than individual query duration alone.
FR-004	Functional	The system shall compile a weekly performance optimization digest summarizing the top slow queries, open index recommendations, and week-over-week latency trends, and make it available to the Backend Lead.	High	Pass: A digest is generated every 7 days and contains all three sections when relevant data exists. Fail: The digest is missing a required section or is not generated according to the configured schedule.	Gives the Backend Lead a periodic, low-effort overview of database health without requiring manual log analysis.
FR-005	Functional	The system shall allow a Database Administrator to mark an index recommendation as Applied, Dismissed, or Pending, and shall exclude Applied or Dismissed recommendations from future digests.	Medium	Pass: Marking a recommendation as Applied removes it from the next generated digest. Fail: A recommendation still appears after being marked as Applied.	Prevents repeated alerts and allows the team to track remediation status over time.
NFR-001	Non-Functional	Slow query log ingestion must handle up to 5,000 log records per minute without dropping entries	High	Pass: Benchmarking tests confirm that the system processes 5,000	Ensures the observability system can operate at production scale

		or slowing the host database.		log records per minute without record loss and meets the defined latency requirements under simulated peak load. Fail: Records are dropped, throughput falls below the target, or the host database experiences unacceptable performance degradation.	without negatively affecting the database it is monitoring.
NFR-002	Non-Functional	The system shall redact literal values, such as values in WHERE clause conditions, from captured SQL text before storage or display, while retaining the parameterized query structure.	High	Pass: Stored or displayed queries contain placeholders instead of literal values for 100% of sampled captured queries during security review. Fail: Any literal value is visible in stored or displayed query text.	Slow query logs may contain sensitive information such as email addresses or IDs. Redaction prevents accidental exposure through the profiler's storage or user interface.

2. UML Use-Case Diagram (PDF), showing all actors, use cases, and at least one «include» or «extend» relation.



3. Use-Case Flow Specification : Database Query Performance Profiler

PES1UG24CS082 | Problem Statement #44

Use Case: UC-03 : Identify Missing Index Candidates

Primary Actor: Database Administrator (via system, triggered after UC-02: Parse Execution Plan)

Related Requirement: FR-001

Preconditions

1. The target slow query has already been ingested (UC-01) and its EXPLAIN plan has been successfully parsed into a structured node tree (UC-02).
2. The Database Administrator has valid, active credentials and read access to the profiler for the relevant database instance.
3. Table and index metadata (schema, existing indexes, row-count estimates) for the queried tables is available to the profiler.

Postconditions

- Success: One or more ranked index recommendations are attached to the query record, each with the candidate columns, estimated impact, and status set to "Pending," and the query becomes eligible for inclusion in the next weekly digest (UC-04).
- Failure: The query record is flagged "No index candidate found" and no recommendation is created; the DBA is not interrupted.

Main Success Scenario

1. The system selects the next parsed execution plan awaiting index analysis from the queue.
2. The system walks the plan's node tree and isolates nodes marked as sequential/table scans on tables above the configured row-count threshold.
3. For each flagged scan node, the system extracts the filter, join, and sort columns referenced at that node.
4. The system cross-checks the extracted columns against the table's existing index definitions to rule out columns already covered.

5. The system generates one or more candidate composite index definitions (column order based on selectivity and filter usage) for the remaining uncovered columns.
6. The system estimates the impact of each candidate (e.g., projected scan-to-seek conversion, estimated row reduction) using available table statistics.
7. The system ranks the candidates by estimated impact and attaches the top candidate(s) to the query record with status "Pending."
8. The system marks the query record as "Analyzed" and makes it visible to the Database Administrator in the recommendations view.
9. Use case ends successfully.

Alternate Flow — 6a: No Table Statistics Available for Impact Estimation

- 6a1. If table statistics needed for impact estimation are missing or stale (not refreshed within the configured window), the system still generates the candidate index definition(s) from step 5.
- 6a2. The system marks the recommendation's estimated impact as "Unavailable" instead of a numeric value and appends a note requesting a statistics refresh.
- 6a3. Flow resumes at step 7, ranking statistics-backed candidates above unavailable-impact candidates.